



UNIVERSIDAD DE LAS FUERZAS ARMADAS ESPE

INGENIERÍA EN SOFTWARE

Analisis y Diseño de Software
30724

Presenta:

Altamirano Xavier

Ayo Alisson

Cordero Martin

Grupo: 1

Fecha:

21 de Mayo de 2026



 FORMATO PARA ELABORACIÓN DE TALLERES ESTUDIANTILES	Departamento de Ciencias de la Computación
	Carrera: Ingeniería de Software Asignatura: Análisis y diseño de software NRC: 30724
	Ayo Alisson, Altamirano Xavier, Cordero Martin

1. Objetivo

Analizar e implementar los patrones de diseño Abstract Factory, Singleton y Factory Method en el desarrollo de un CRUD de estudiantes, aplicando conceptos de análisis y diseño de software para mejorar la organización, reutilización, mantenimiento y escalabilidad de la aplicación.

2. Desarrollo

2.1. Patrones creacionales

2.1.1. Singleton

Descripción:

El Singleton es un patrón de diseño de tipo creacional cuyo objetivo principal es que una clase tenga una única instancia. Este patrón es especialmente útil cuando se necesita exactamente un objeto para coordinar acciones a través de un sistema, como en el caso de gestores de bases de datos, sistemas de registro como logging y el manejo de archivos.

Además de garantizar la unicidad, el Singleton proporciona un punto de acceso global a dicha instancia, permitiendo que sea alcanzable desde cualquier parte del programa sin el riesgo de que el objeto sea sobrescrito por otro código

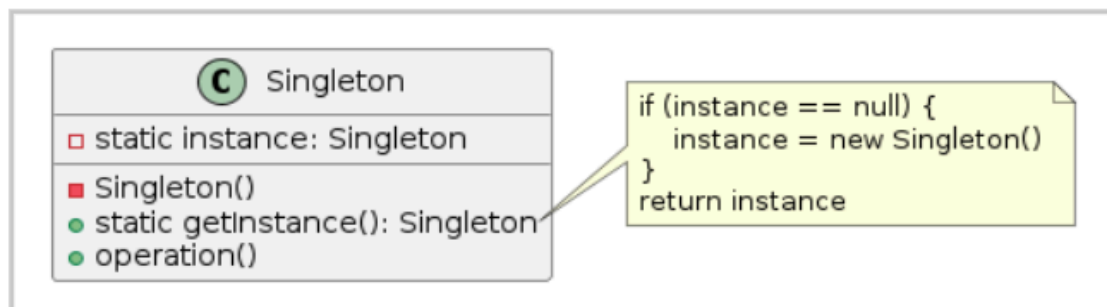
Características

- **Instancia Única:** Garantiza que no existan múltiples copias de la misma clase en memoria.

 FORMATO PARA ELABORACIÓN DE TALLERES ESTUDIANTILES	Departamento de Ciencias de la Computación
	Carrera: Ingeniería de Software Asignatura: Análisis y diseño de software NRC: 30724
	Ayo Alisson, Altamirano Xavier, Cordero Martin

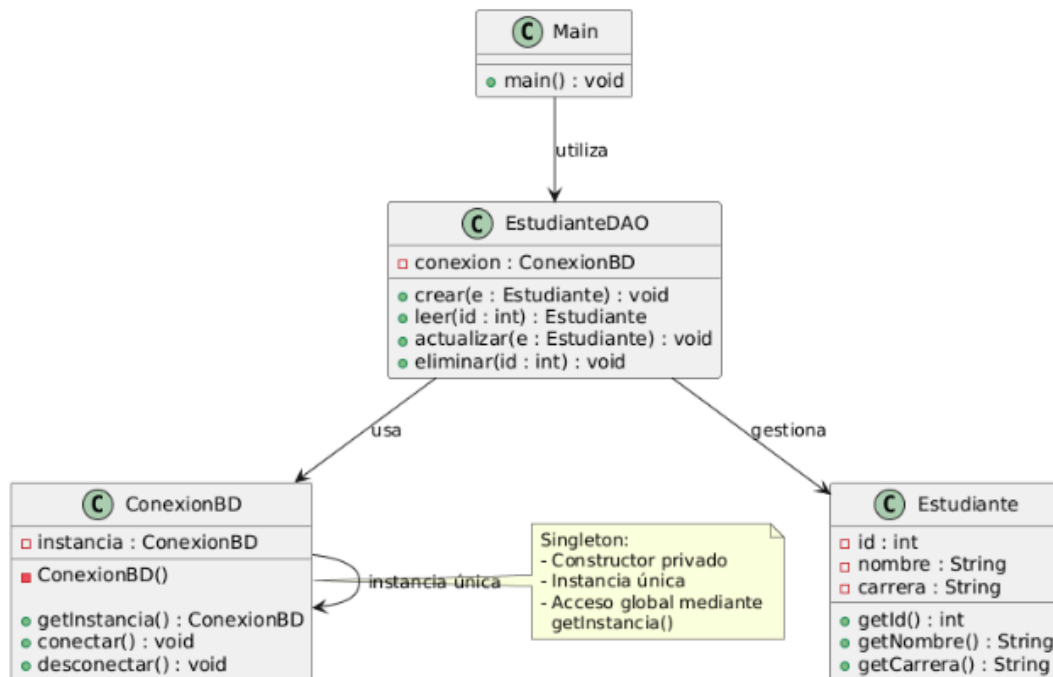
- Constructor Privado: Se oculta el constructor por defecto para evitar que otros objetos utilicen el operador new.
- Método Estático de Creación: Incluye un método (comúnmente llamado getInstance) que actúa como constructor especial: crea el objeto en la primera llamada y lo devuelve en las llamadas subsiguientes.
- Inicialización Diferida (Lazy Initialization): El objeto solo se crea en el momento en que se solicita por primera vez, ahorrando recursos del sistema.
- Control de Acceso Global: Actúa de forma similar a una variable global, pero con una protección que impide que la instancia sea sustituida

Diagrama de clases:



 FORMATO PARA ELABORACIÓN DE TALLERES ESTUDIANTILES	Departamento de Ciencias de la Computación
	Carrera: Ingeniería de Software Asignatura: Análisis y diseño de software NRC: 30724
	Ayo Alisson, Altamirano Xavier, Cordero Martin

Patrón Singleton aplicado a CRUD de Estudiantes



Ventajas y desventajas:

Ventajas

- **Certeza de Instancia Única:** Se tiene el control total sobre el número de instancias disponibles.
- **Acceso Global:** Facilita el uso del objeto desde cualquier módulo del sistema sin pasar referencias constantemente.

 FORMATO PARA ELABORACIÓN DE TALLERES ESTUDIANTILES	Departamento de Ciencias de la Computación
	Carrera: Ingeniería de Software Asignatura: Análisis y diseño de software NRC: 30724
	Ayo Alisson, Altamirano Xavier, Cordero Martin

- Eficiencia de Memoria: Al compartir una única instancia y usar inicialización diferida, se optimiza el uso de memoria comparado con variables globales que consumen recursos desde el inicio.
- Base para otros patrones: Sirve como fundamento para patrones como *Abstract Factory*, *Builder* o *Prototype*.

Desventajas:

- **Vulneración del SRP:** El patrón resuelve dos problemas a la vez (unicidad y acceso global), violando el **Principio de Responsabilidad Única**.
- **Acoplamiento y Mal Diseño:** Puede enmascarar un diseño deficiente donde los componentes dependen demasiado de un estado global oculto.
- **Dificultad en Pruebas Unitarias:** Muchos frameworks de prueba dependen de la herencia para crear objetos simulados (*mocks*). Dado que el Singleton tiene un constructor privado, es difícil de sustituir, lo que complica el testeo.
- **Problemas en Multihilo:** En entornos con múltiples hilos, requiere lógica adicional compleja para evitar que se creen varias instancias simultáneamente

Evidencias de Implementación

 FORMATO PARA ELABORACIÓN DE TALLERES ESTUDIANTILES	Departamento de Ciencias de la Computación
	Carrera: Ingeniería de Software Asignatura: Análisis y diseño de software NRC: 30724
	Ayo Alisson, Altamirano Xavier, Cordero Martin

```

src > modelo > J model.java > model > getInstance()
1  package modelo;
2
3  import factory.EstudianteFactory;
4  import java.util.ArrayList;
5  import java.util.List;
6
7  public class model {
8      private static final model INSTANCIA = new model();
9      private final List<Estudiante> estudiantes;
10
11     private model() {
12         this.estudiantes = new ArrayList<>();
13     }
14
15     public static model getInstance() {
16         return INSTANCIA;
17     }
18

```

como funciona?

```

public class model {
    // ← AQUÍ: Instancia estática única (privada)
    private static final model INSTANCIA = new model();

    // Lista de estudiantes
    private final List<Estudiante> estudiantes;

    // Constructor privado (NO puede crear instancias nuevas)
    private model() {
        this.estudiantes = new ArrayList<>();
    }

    // ← AQUÍ: Método para obtener la única instancia
    public static model getInstance() {
        return INSTANCIA;
    }
}

```

	Departamento de Ciencias de la Computación
	Carrera: Ingeniería de Software Asignatura: Análisis y diseño de software NRC: 30724
	Ayo Alisson, Altamirano Xavier, Cordero Martin

```

J App.class      J App.java M X
src > J App.java > ...
1  import controlador.controller;
2  import factory.ModernCrudUIFactory;
3  import modelo.model;
4  import vista.view;
5
6  public class App {
7      Run | Debug
8      public static void main(String[] args) throws Exception {
9          // Singleton: el modelo se obtiene desde una única instancia compartida
10         model miModelo = model.getInstance();

```

2.1.2. Factory Method

Descripción:

Factory Method es un patrón de diseño de tipo creacional, este define una interfaz para la creación de objetos en una superclase, lo que permite que las subclasses modifiquen simultáneamente el tipo de objetos que se crearán, en lugar de instanciar objetos mediante el operador new, este patrón propone la innovación de un método llamado “fábrica”.

Su objetivo principal es suavizar el acoplamiento entre una clase y los objetos que instancia. Este diseño funciona delegando la responsabilidad de la instanciación a las subclasses, de tal manera que la base del creador puede trabajar con una interfaz producto genérica, mientras que las subclasses deciden qué implementación se debe producir según el contexto.

Características

 FORMATO PARA ELABORACIÓN DE TALLERES ESTUDIANTILES	Departamento de Ciencias de la Computación
	Carrera: Ingeniería de Software Asignatura: Análisis y diseño de software NRC: 30724
	Ayo Alisson, Altamirano Xavier, Cordero Martin

Sus principales características se centran en la flexibilidad y la adherencia a las buenas prácticas de ingeniería de software.

Interfaz Común para Productos: Todos los objetos devueltos por el método fábrica deben seguir una interfaz o clase base común, lo que permite que el código cliente trate a los productos de forma polimórfica.

Encapsulamiento de la Creación: Centraliza la lógica de construcción de objetos en un solo lugar, facilitando el mantenimiento y la evolución del sistema.

Principio de Responsabilidad Única (SRP): Permite mover el código de creación de productos a un lugar específico del programa, evitando que las clases de lógica de negocio se saturen con detalles de instanciación.

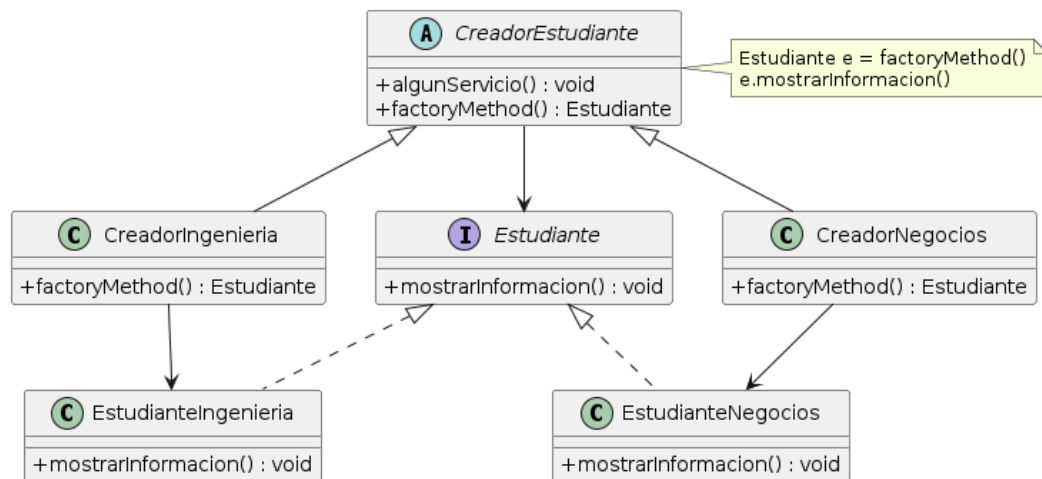
Principio de Abierto/Cerrado (OCP): Es posible introducir nuevos tipos de productos en la aplicación sin necesidad de modificar el código cliente existente, simplemente creando nuevas subclases creadoras.

Desacoplamiento: El código que utiliza el producto (código cliente) no conoce las clases concretas que se instancian, sino que interactúa únicamente con la interfaz abstracta.

Diagrama de clases:

 FORMATO PARA ELABORACIÓN DE TALLERES ESTUDIANTILES	Departamento de Ciencias de la Computación
	Carrera: Ingeniería de Software Asignatura: Análisis y diseño de software NRC: 30724
	Ayo Alisson, Altamirano Xavier, Cordero Martin

Factory Method - CRUD de Estudiantes



Ventajas y desventajas:

Ventajas

- Evita el acoplamiento fuerte entre el creador y los productos concretos.
- Facilita la extensión del código (Principio de Abierto/Cerrado), permitiendo añadir productos sin romper el sistema.
- Centraliza la creación de objetos, lo que simplifica el mantenimiento y la aplicación del Principio de Responsabilidad Única.
- Permite la reutilización de recursos, como conexiones de bases de datos o sistemas de archivos, al poder devolver objetos existentes de una caché en lugar de crear siempre nuevos.

Desventajas

 FORMATO PARA ELABORACIÓN DE TALLERES ESTUDIANTILES	Departamento de Ciencias de la Computación
	Carrera: Ingeniería de Software Asignatura: Análisis y diseño de software NRC: 30724
	Ayo Alisson, Altamirano Xavier, Cordero Martin

- Aumento de la complejidad del código, ya que requiere la creación de múltiples subclases nuevas para cada tipo de producto.
- Dificultad de lectura inicial para desarrolladores no familiarizados con patrones, el flujo de creación puede parecer indirecto o innecesariamente complicado.

Evidencias de implementación

```

J EstudianteFactory.java X
src > factory > J EstudianteFactory.java
1  package factory;
2
3  import modelo.model.Estudiante;
4
5  public final class EstudianteFactory {
6      private EstudianteFactory() {
7      }
8
9      public static Estudiante crearEstudiante(int id, String nombre, int edad) {
10         return new Estudiante(id, nombre.trim(), edad);
11     }

```

```

// Agregar estudiante
public boolean agregarEstudiante(int id, String nombre, int edad) {
    if (!validarID(id) || !validarNombre(nombre) || !validarEdad(edad)) {
        return false;
    }
    if (idExiste(id)) {
        return false;
    }
    estudiantes.add(EstudianteFactory.crearEstudiante(id, nombre, edad));
    return true;
}

```

 FORMATO PARA ELABORACIÓN DE TALLERES ESTUDIANTILES	Departamento de Ciencias de la Computación
	Carrera: Ingeniería de Software Asignatura: Análisis y diseño de software NRC: 30724
	Ayo Alisson, Altamirano Xavier, Cordero Martin

```

JPanel panel = uiFactory.crearPanelSeccion();
panel.setLayout(new FlowLayout(FlowLayout.CENTER, hgap: 15, vgap: 10));
panel.setBorder(uiFactory.crearBordeTitulo(titulo: "Acciones"));

// Botón Agregar
btnAgregar = uiFactory.crearBoton(texto: "Agregar Estudiante");
btnAgregar.addActionListener(e -> agregarEstudiante());
panel.add(btnAgregar);

```

2.1.3. Abstract Factory

Descripción:

El patrón Abstract Factory proporciona una interfaz para crear familias de objetos relacionados o que dependen entre sí, sin necesidad de especificar sus clases concretas. En lugar de crear objetos directamente usando la palabra clave new, el código cliente delega la creación a un objeto fábrica que se encarga de instanciar los productos adecuados.

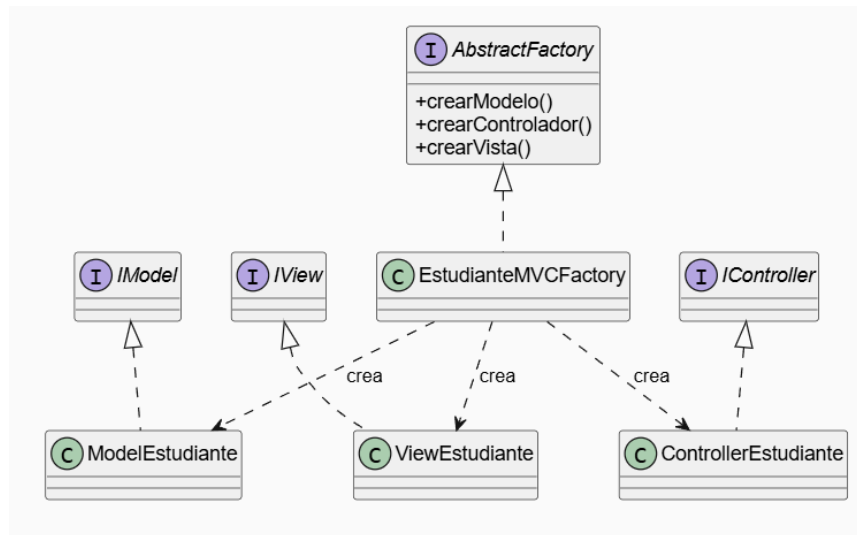
Características:

- **Creación de familias de productos:** Se utiliza cuando un sistema debe ser configurado con una de múltiples familias de productos (por ejemplo, elementos de interfaz gráfica como botones y barras de desplazamiento que deben verse de manera consistente en un entorno Windows o Mac).
- **Abstracción:** Separa los detalles de implementación de las clases concretas (la fábrica concreta) de la interfaz de la que dependen los clientes (la fábrica abstracta).

 FORMATO PARA ELABORACIÓN DE TALLERES ESTUDIANTILES	Departamento de Ciencias de la Computación
	Carrera: Ingeniería de Software Asignatura: Análisis y diseño de software NRC: 30724
	Ayo Alisson, Altamirano Xavier, Cordero Martin

- **Composición de objetos:** Favorece la composición sobre la herencia para la creación de los objetos, delegando la responsabilidad a una o más fábricas.

Diagrama de clases:



En este CRUD de estudiantes, el patrón Abstract Factory se utiliza para crear de manera organizada los componentes principales del sistema MVC: el modelo, el controlador y la vista.

La clase **EstudianteMVCFactory** actúa como una fábrica que genera todos los objetos necesarios para que el CRUD funcione correctamente. De esta manera, la aplicación no necesita crear directamente las clases concretas, sino que delega esa responsabilidad a la fábrica.

Por ejemplo:

	Departamento de Ciencias de la Computación
	Carrera: Ingeniería de Software Asignatura: Análisis y diseño de software NRC: 30724
	Ayo Alisson, Altamirano Xavier, Cordero Martin

- el modelo administra los datos de los estudiantes,
- el controlador procesa las operaciones CRUD,
- y la vista muestra la interfaz gráfica al usuario.

Este patrón ayuda a mantener el sistema más ordenado, flexible y fácil de mantener, ya que separa la lógica de creación de objetos del funcionamiento principal de la aplicación.

Ventajas y desventajas:

Ventajas	Desventajas
Aislamiento de clases concretas: El cliente interactúa únicamente con interfaces abstractas y no necesita conocer los detalles de las clases concretas que se instancian, reduciendo el acoplamiento.	Dificultad para añadir nuevos tipos de productos: Si necesitas añadir un nuevo tipo de producto a la familia, deberás modificar la interfaz de la fábrica abstracta, lo que implica tener que modificar todas las clases de fábricas concretas existentes, rompiendo el principio de Abierto/Cerrado (Open/Closed Principle).
Intercambio fácil de familias de productos: Cambiar la familia entera de productos que utiliza un sistema es tan fácil como cambiar la instancia de la fábrica concreta en un solo lugar del código.	Complejidad adicional: Aumenta considerablemente el número de interfaces y clases en el sistema (patrón de muchas clases abstractas y concretas), lo que puede complicar el código fuente más de lo necesario en proyectos pequeños o sencillos.

 FORMATO PARA ELABORACIÓN DE TALLERES ESTUDIANTILES	Departamento de Ciencias de la Computación
	Carrera: Ingeniería de Software Asignatura: Análisis y diseño de software NRC: 30724
	Ayo Alisson, Altamirano Xavier, Cordero Martin

Consistencia garantizada: Asegura que los productos generados por una fábrica concreta estén diseñados para trabajar juntos, evitando que el cliente mezcle objetos incompatibles.

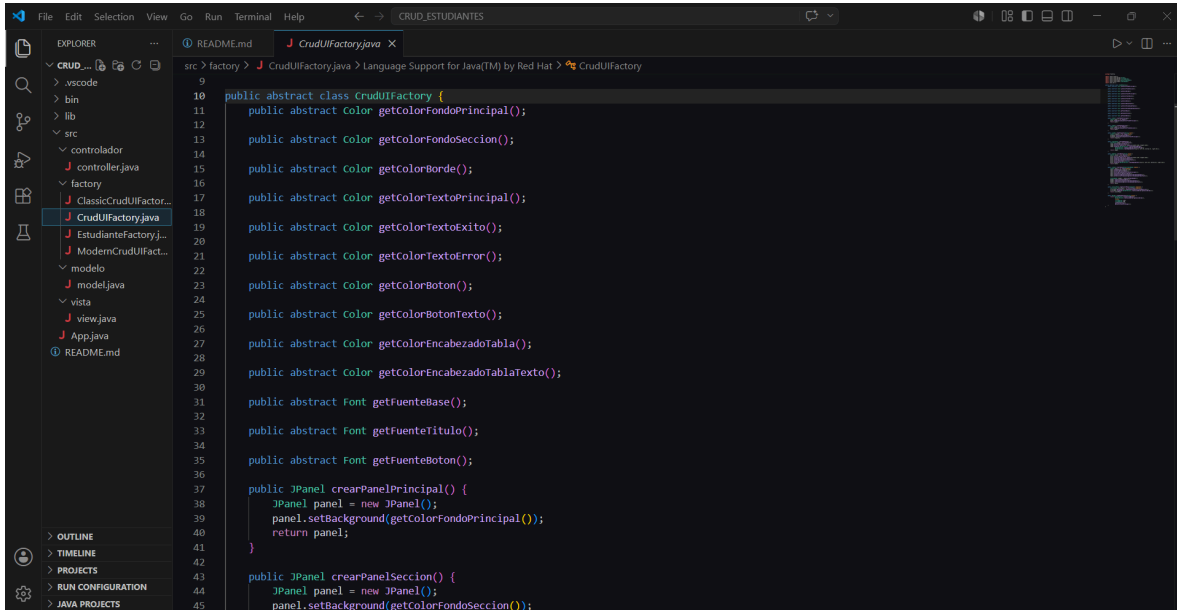
Evidencias de la aplicación de Abstract Factory

```

9
10 public abstract class CrudUIFactory {
11     public abstract Color getColorFondoPrincipal();
12
13     public abstract Color getColorFondoSeccion();
14
15     public abstract Color getColorBorde();
16
17     public abstract Color getColorTextoPrincipal();
18
19     public abstract Color getColorTextoExito();
20
21     public abstract Color getColorTextoError();
22
23     public abstract Color getColorBoton();
24
25     public abstract Color getColorBotonTexto();
26
27     public abstract Color getColorEncabezadoTabla();
28
29     public abstract Color getColorEncabezadoTablaTexto();
30
31     public abstract Font getFuenteBase();
32
33     public abstract Font getFuenteTitulo();
34
35     public abstract Font getFuenteBoton();
36
37     public JPanel crearPanelPrincipal() {
38         JPanel panel = new JPanel();
39         panel.setBackground(getColorFondoPrincipal());
40         return panel;
41     }
42
43     public JPanel crearPanelSeccion() {
44         JPanel panel = new JPanel();
45         panel.setBackground(getColorFondoSeccion());

```

 FORMATO PARA ELABORACIÓN DE TALLERES ESTUDIANTILES	Departamento de Ciencias de la Computación
	Carrera: Ingeniería de Software Asignatura: Análisis y diseño de software NRC: 30724
	Ayo Alisson, Altamirano Xavier, Cordero Martin



```

src > factory > J CrudUIFactory.java > Language Support for Java(TM) by Red Hat > CrudUIFactory
9
10 public abstract class CrudUIFactory {
11     public abstract Color getColorFondoPrincipal();
12     public abstract Color getColorFondoSeccion();
13     public abstract Color getColorBorde();
14     public abstract Color getColorTextoPrincipal();
15     public abstract Color getColorTextoExito();
16     public abstract Color getColorTextoError();
17     public abstract Color getColorBoton();
18     public abstract Color getColorBotonTexto();
19     public abstract Color getColorEncabezadoTabla();
20     public abstract Color getColorEncabezadoTablaTexto();
21     public abstract Font getFuenteBase();
22     public abstract Font getFuenteTitulo();
23     public abstract Font getFuenteBoton();
24     public JPanel crearPanelPrincipal() {
25         JPanel panel = new JPanel();
26         panel.setBackground(getColorFondoPrincipal());
27         return panel;
28     }
29     public JPanel crearPanelSeccion() {
30         JPanel panel = new JPanel();
31         panel.setBackground(getColorFondoSeccion());
32     }
33 }

```

3. Conclusión técnica

Consideramos que la implementación de los patrones de diseño Singleton, Factory Method y Abstract Factory dentro del CRUD de estudiantes permitió comprender de manera práctica cómo estos patrones ayudan a mejorar la estructura y organización del software. Considero que el uso de estos patrones facilita la separación de responsabilidades, reduce el acoplamiento entre clases y permite que el sistema sea más flexible y mantenible a largo plazo.

Además, la aplicación de Abstract Factory permitió organizar correctamente los componentes principales del sistema, mientras que Factory Method ayudó a encapsular la creación de objetos y Singleton garantizó una única instancia para determinados recursos del sistema. En conjunto, estos patrones aportan buenas prácticas de ingeniería de software

	Departamento de Ciencias de la Computación
	Carrera: Ingeniería de Software Asignatura: Análisis y diseño de software NRC: 30724
	Ayo Alisson, Altamirano Xavier, Cordero Martin

y preparan la aplicación para futuras ampliaciones sin afectar significativamente su funcionamiento actual.

4. Recomendaciones

- Implementar pruebas unitarias y de integración para verificar el correcto funcionamiento de cada patrón aplicado, especialmente en clases Singleton y fábricas, con el fin de asegurar la estabilidad y mantenibilidad del sistema.
- Continuar profundizando en otros patrones de diseño y arquitectónicos, como MVVM o Repository, para complementar el desarrollo del CRUD y mejorar la escalabilidad, reutilización y organización del proyecto en aplicaciones más complejas.

Preguntas

Pregunta 1. Singleton y consistencia del estado

En un CRUD de Estudiante, explique en qué escenario sería pertinente usar Singleton para gestionar una única instancia del repositorio o conexión. Analice también un riesgo de acoplamiento o dificultad de pruebas unitarias que podría generar su uso.

El patrón Singleton es útil cuando necesitas un único punto de acceso para recursos compartidos, pero conlleva importantes riesgos arquitectónicos.

La gestión de Base de Datos centraliza y controla el acceso a la base de datos evitando el agotamiento de recursos o fallos de concurrencia.

También se asegura que todas las capas del sistema interactúen con la misma lista de datos, evitando desincronizaciones.

 FORMATO PARA ELABORACIÓN DE TALLERES ESTUDIANTILES	Departamento de Ciencias de la Computación
	Carrera: Ingeniería de Software Asignatura: Análisis y diseño de software NRC: 30724
	Ayo Alisson, Altamirano Xavier, Cordero Martin

Pero también tenemos una desventaja ya que las clases quedan rígidamente atadas a una implementación específica. Cambiar de base de datos o de sistema de almacenamiento requerirá modificar mucho código. Y al introducir un estado global, los tests pueden contaminarse entre sí. Además, hace que sea muy difícil usar objetos de prueba, lo que resulta en pruebas frágiles y lentas.

Pregunta 2. Factory Method y creación de estudiantes

Compare la creación directa de un objeto Estudiante mediante new con el uso de Factory Method. ¿Cómo cambia la responsabilidad del controlador y qué beneficio tendría si luego se agregan tipos como EstudianteIngeniería, EstudianteNegocios, becado o intercambio?

Al crear un nuevo objeto mediante la palabra new, el controlador es responsable de conocer y crear las clases lo que genera un mayor acoplamiento lo cual hace que el código sea difícil de modificar. En cambio en el patrón Factory Method la responsabilidad la tiene una clase factory lo que permite que el controlador solo solicite el objeto. Este patrón facilita la implementación de nuevos tipos como EstudianteIngeniería, EstudianteNegocios, becado o de intercambio, sin tener que modificar constantemente el controlador haciendo el sistema más flexible y fácil de mantener.

 FORMATO PARA ELABORACIÓN DE TALLERES ESTUDIANTILES	Departamento de Ciencias de la Computación
	Carrera: Ingeniería de Software Asignatura: Análisis y diseño de software NRC: 30724
	Ayo Alisson, Altamirano Xavier, Cordero Martin

Pregunta 3. Abstract Factory frente a Factory Method

Justifique cuándo sería más adecuado aplicar Abstract Factory en lugar de Factory Method dentro del CRUD. Considere la creación de familias de componentes relacionados, por ejemplo modelo, vista y controlador, y explique cómo esto impacta en la coherencia y escalabilidad del sistema.

Abstract factory es más adecuado que factory method cuando se necesita crear familias de objetos relacionados, por ejemplo, distintos modelos, vistas y controladores cohesivos, asegurando que los objetos creados sean compatibles entre sí y permitiendo intercambiar fácilmente implementaciones concretas de toda una familia.

Factory es ideal cuando solo se necesita variar la creación de un solo tipo de producto (por ejemplo, solo diferentes modelos de estudiante) y cuando las clases creadoras pueden funcionar independientemente unas de otras.

Abstract Factory, en cambio, define una interfaz para crear un conjunto o familia de objetos relacionados sin especificar sus clases concretas. Esto es útil cuando el sistema debe ser independiente de cómo se crean, componen y representan sus productos, y se requiere consistencia al usar productos de la misma familia.